

11-2 Advanced Consumer Analysis and CRM in Marketing

Chapter Introduction

In this chapter, we will learn a slightly more advanced approach to marketing analysis than in week 11-1. The goal of this lecture is to propose better services and products to customers, efficiently execute and verify corporate initiatives, and automate the reporting of the results of those initiatives and KPIs. We will introduce tools that will help build systems for automating marketing in the future.

In week 11-1, we segmented customers and discussed the need for actions tailored to each customer group. In this advanced section, we will focus on “Recommendation Systems” to suggest services that customers are likely to buy, “Causal Inference and Analysis” to judge whether the initiatives implemented by the company were successful, and “Automation” for tracking results and KPIs, creating regular analytical reports, automating email dispatch, and sending automatic notifications to Slack, etc.

These are deep fields, and it is difficult to cover all of them in this chapter. However, starting from the marketing strategies learned in week 11-1, you will be able to imagine the entire basic flow from consumer analysis, segmentation and targeting, to actions and verification for customers, and the automation and reporting of those processes. When you need to dive deeper into each area in practice, refer to the resources introduced later to get started.

Goal

Understand advanced marketing approaches (recommendations, causal inference, and automated analysis reports), be able to implement the basics of each, and design systems to operate the PDCA cycle in marketing.

Prerequisite Knowledge and Skills

A thorough understanding and ability to implement the week 11-1 of consumer analysis and CRM in marketing.

✓ 11-2.1 Background and Overview of This Chapter

Keywords: Recommendation, Causal Inference, Marketing System Automation

✓ 11-2.1.1 Customer Proposals, Campaign Evaluation, and Automation

In the Introduction to Consumer Analysis and CRM in Marketing, we covered basic customer behavior analysis and how to handle POS data, which is part of CRM. Until now, we have focused on learning the fundamental methods of data analysis, including basic aggregation, examining raw data when necessary, identifying (segmenting) customers in marketing through various data analysis techniques, and how to apply these methods. The next steps to consider are:

"What products or services should be offered or proposed to these customer segments?" "Did the campaign have an effect? I want to fully understand the ROI and evaluate it." "I want to automate the KPI tracking reports for such campaigns, rather than doing them manually."

These involve creating more concrete strategies, actions for customers, measuring the effectiveness of these strategies, and automating report generation.

The following points will be covered in this lecture:

1. Recommendation (Recommendation System)
2. Campaign Effectiveness Evaluation (often referred to as Test & Learn)
3. BI Tool Creation and Report Automation

As practiced in the Introduction to Consumer Analysis and CRM in Marketing, data analysis often involves meticulously examining data and performing many tedious tasks. However, of course, we do not want to continue such tasks forever. Instead, we aim to build an optimal "algorithm-driven automated marketing ecosystem" for customers, letting the system handle what it can and freeing up more time for creative tasks or work that only humans can do. In this chapter, we will learn approaches and tools for this automation.

Note: As for marketing automation tools, there are various solutions like Marketo for Marketing Automation (MA), which have become widely used by many companies. On the other hand, there are also cases where these tools are underutilized, with companies simply implementing them without fully leveraging their capabilities. These tools are just that—tools—and are not all-powerful. It is important to first clarify the purpose, such as

how they will contribute to improving customer service or reducing operational costs, before implementing them.

✓ 11-2.1.2 Libraries Mainly Used in This Chapter

Here are the libraries required for this chapter. If any additional libraries are necessary, they will be imported as needed.

```
# Load the following libraries in advance as they will be used in this chapter
import numpy as np
import pandas as pd

import datetime

# Visualization libraries
import matplotlib.pyplot as plt
import seaborn as sns
sns.set()
%matplotlib inline

# Display up to 3 decimal places
%precision 3

'%.3f'
```

✓ 11-2.2 Recommendation

Keywords: Collaborative Filtering, Content-Based Filtering, SVD, Boltzmann Machine, Reinforcement Learning

✓ 11-2.2.1 What you will learn in this chapter

In the modern era, it is said that we are living in an age of information overload. While customers have many choices, they often don't know which products or services to choose. On the other hand, when companies recommend products to customers, simply recommending the top-ranking products will not help in suggesting products that are truly suited to the customer, and opportunities to sell niche products may also decrease. Many people are likely familiar with the concept of recommendations, as they are used by well-known services like Amazon, Netflix, and Youtube to display suggested products or content.

Note: When a customer clearly knows what product they are looking for, they can simply "search" for it, while recommendations are used when the customer has hidden needs that they may not be able to express verbally. (When recommendations are based on clear user intent, such as during a search, it's referred to as a "pull model." On the other hand, when recommendations are without clear intent, it's called a "push model.")

Recommendation systems are mainly calculated based on customer ratings for products, their purchase history (POS data), and other factors. There are various ways to categorize types of recommendations, but there are primarily three approaches: an approach that searches for similar customers (**collaborative filtering**), an approach based on similar products (**content-based filtering**), and an approach based on context (**context-based filtering**). The first two are somewhat easy to imagine, but the third involves context. For example, the same customer may tend to buy different products depending on the season, so the context in this case would be the season.

- **Collaborative Filtering:** Recommendations based on similar people
- **Content-based Filtering:** Recommendations based on the content of the products
- **Context-based Filtering:** Recommendations based on the customer's environment (location, time, season, channel, conditions, etc.)

In this section, we will primarily introduce the first type of recommendation engine, "Collaborative Filtering," and its implementation in Python. For those who would like to learn more about the different types of recommendations and the underlying logic, please refer to the following references.

References

- [AI Algorithm Marketing: Machine Learning/Economic Models, Best Practices, and Architecture] AI Arugorizumu Māketingu Jidōka no tame no Kikai Gakushū/Ekonomī Moderu, Besuto Purakutisu, Ākitekuchā (in Japanese)
- [Recommendation Systems] Suisen Shisutemu (in Japanese)
- [Introduction to Information Recommendation Systems] Jōhō Suisen Shisutemu Nyūmon (in Japanese)

Additionally, the following publicly available materials are highly recommended for learning about recommendation systems:

Reference URLs

- Algorithms for Recommendation Systems by Toshihiro Kamishima
- <https://www.kamishima.net/archive/recsysdoc.pdf>
- <https://www.kamishima.net/archive/recsys.pdf>

✓ 11-2.2.2 Collaborative Filtering

From here, we will introduce the Python implementation of collaborative filtering. As explained above, collaborative filtering is a recommendation method based on similar users. In other words, it involves finding customers whose preferences are similar to the target customer, and then making recommendations based on the preferences of those similar customers. This method is widely adopted in the recommendation systems of various companies.

To implement that algorithm, we will use the scikit-surprise package here. The installation will be executed as follows.

Reference: Official website of the scikit-surprise package

- <http://surpriselib.com/>
- <https://surprise.readthedocs.io/en/stable/index.html#>

```
!pip install scikit-surprise
```

```
Collecting scikit-surprise
  Downloading scikit_surprise-1.1.4.tar.gz (154 kB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 154.4/154.4 kB 4.8 MB/s eta 0:00:00
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.10/dist-p
Requirement already satisfied: numpy>=1.19.5 in /usr/local/lib/python3.10/dist-p
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.10/dist-pa
Building wheels for collected packages: scikit-surprise
  Building wheel for scikit-surprise (pyproject.toml) ... done
  Created wheel for scikit-surprise: filename=scikit_surprise-1.1.4-cp310-cp310-
  Stored in directory: /root/.cache/pip/wheels/4b/3f/df/6acbf0a40397d9bf3ff97f58
Successfully built scikit-surprise
Installing collected packages: scikit-surprise
Successfully installed scikit-surprise-1.1.4
```

Next, we will import the libraries necessary to implement and validate this collaborative filtering algorithm. SVD (Singular Value Decomposition) is a commonly used algorithm in

recommendation systems. `Cross_validate` is a library frequently used for model evaluation in machine learning, and we will import it to perform cross-validation.

```
from surprise import SVD
from surprise.model_selection import cross_validate
from surprise import Dataset
```

The final dataset is used to import the data we need here, which consists of users' movie ratings. It will ask you, "Do you want to download it? [Y/n]", so enter "Y" and press Enter to load the movielens-100k dataset.

```
movie_dataset = Dataset.load_builtin('ml-100k')
```

```
Dataset ml-100k could not be found. Do you want to download it? [Y/n]
```

The details about this dataset can be found below. The "100k" refers to 100,000 records.

Reference URLs:

- <https://grouplens.org/datasets/movielens/>
- <https://surprise.readthedocs.io/en/stable/dataset.html>

You can access the data with `raw_ratings`, which is a list of data, where each element is a tuple containing individual data entries.

```
type(movie_dataset.raw_ratings)
```

From left to right, the data consists of the following: User ID, Movie ID of the rated film, Rating given to the film, and the Timestamp of when the rating was made. Each tuple represents one user's rating for a single movie.

```
movie_dataset.raw_ratings[0:10]
```

If you want to handle the data as a pandas DataFrame, you can convert it as follows:

```
movie_df = pd.DataFrame(movie_dataset.raw_ratings, columns=['user_id', 'product_
movie_df.head()
```

The timestamp on the far right is not in a readable date and time format, so let's convert it into a more understandable datetime format. The unit argument for pandas' `to_datetime` function uses 's' to represent seconds.

```
movie_df['timestamp'] = pd.to_datetime(movie_df.timestamp.astype(int), unit='s')
```

```
movie_df.head()
```

```
movie_df.info()
```

As seen above, the timestamp has been properly converted to the date type. Additionally, we can confirm from the info that there are 100,000 rows of data. Now, the goal here is to build a recommendation algorithm using this data, but before that, let's quickly aggregate the data. From the following, we can see that there are 943 users who have given ratings, and 1682 products. It is also clear that the average ratings differ by product.

```
movie_df.user_id.nunique()
```

```
movie_df.product_id.nunique()
```

```
movie_df.groupby(["product_id"])["rating"].mean().sort_values()
```

How many products has each person rated? From the following, we can see that in this data, each person has rated between 20 and 737 products.

```
movie_df.groupby(["user_id"])["product_id"].nunique().sort_values(ascending=False)
```

Next, when building this recommendation algorithm, it is common to create a two-dimensional table (matrix) showing how each user rates each movie, and the table can be created with the following implementation. (We will use pandas' `pivot_table`.)

```
movie_df.pivot_table(index='user_id', columns='product_id', values='rating')
```

As we can see in the table above, this kind of data is sparse, meaning there are many missing values. With 943 users and 1682 items, there should ideally be 1,586,126 ratings. However, in reality, most of the data is not rated.

Now, to build the recommendation system using this data, the target variable will be the rating, and we aim to build a model to validate it. The approach is similar to the validation method learned in machine learning. In other words, we will split the data into training (and validation) data and test data, build the algorithm using the training data, and evaluate it using the test data.

We will use **SVD (Singular Value Decomposition)**, an algorithm commonly introduced in recommendation systems. This is an approach in collaborative filtering (since this lecture focuses on understanding how to use it, the detailed calculations and exact definitions are not discussed here, but those interested can refer to the reference materials).

For model evaluation, we perform cross-validation with $cv=5$. As for evaluation metrics, RMSE and MAE are commonly used, but to emphasize clarity, we will set only RMSE. (You can review about supervised learning using materials from week 5.)

- MSE (Mean Squared Error): The average of the squared differences between the predicted values and the actual values.
- RMSE (Root Mean Squared Error): The square root of MSE.
- MAE (Mean Absolute Error): The average of the absolute differences between the predicted values and the actual values.

The implementation of the SVD model is as follows.

```
algo = SVD()
```

Next, we use `cross_validate`. The arguments include the algorithm to be used, the dataset, evaluation metrics, the number of CV folds, and whether or not to display output at each computation step (Flg). Please note that the execution may take some time. (Also, note that the dataset used here is the `movie_dataset` loaded directly, not the DataFrame we converted earlier.)

```
cross_validate(algo, movie_dataset, measures=['RMSE'], cv=5, verbose=True)
```

From the above results, the average RMSE for the SVD model is approximately 0.93. However, just looking at this result alone does not allow us to determine whether the SVD model is good or not. This is because there is no comparison baseline. As mentioned in

the introductory section, the basics of analysis lie in "comparison." Therefore, let's first build a random model and compare the results with it (if someone is using their own original algorithm for recommendations, that would serve as the comparison).

The following `NormalPredictor` represents the results of a random model. The RMSE for the SVD model was about 0.93, but the RMSE for the `NormalPredictor` is about 1.5, showing that the SVD model performs better. (Since these metrics calculate the error between predicted values and actual values, a model with results closer to 0 is considered better.) Note that `NormalPredictor` is not directly used in the engine, but it is often used as a "comparison baseline" for models. It can be useful when there is no other model to compare against in a recommendation system.

```
from surprise import NormalPredictor
algo = NormalPredictor()
cross_validate(algo, movie_dataset, measures=['RMSE'], cv=5, verbose=True)
```

Other algorithms are available at the following reference site, so let's try them out in the next exercise.

https://surprise.readthedocs.io/en/stable/prediction_algorithms_package.html

<Practice Question 11-2-1>

1. Construct a recommendation algorithm using different models on the above data and evaluate them. Also, try using evaluation metrics other than RMSE.

✓ Answer

The following NMF (Non-negative Matrix Factorization) is also based on matrix factorization, and the algorithm used here shows slightly worse results compared to SVD.

```
from surprise import NMF
algo = NMF()
cross_validate(algo, movie_dataset, measures=['RMSE', 'MAE'], cv=5, verbose=True)
```

11-2.2.3 Content-Based Filtering

Next is the product (content)-based recommendation system. Specifically, this approach recommends a different brand B of domestic beer from the same domestic category to a

customer who purchased a domestic beer of brand A. Since the recommendation is based on the content of the product, it is often an algorithm that can be easily implemented by humans, and it is less commonly used in practice compared to collaborative filtering. The surprise package mentioned earlier does not seem to support content-based filtering recommendations at the moment.

Reference: <http://surpriselib.com/>

Additionally, this content-based approach does not rely on users, so it can be used in situations where there are few users or limited product evaluations.

11-2.2.4 Hybrid Methods, Deep Learning, and Reinforcement Learning for Recommendation

Up until now, we have briefly explained the types of recommendations, focusing on collaborative filtering, content-based recommendations, and context-based recommendations. To address the weaknesses of each method, there are hybrid approaches that combine these techniques. Additionally, there are recommendation methods based on association rules.

In addition, there are advanced recommendation methods, such as recommendation systems using Boltzmann Machines in the field of deep learning (for collaborative filtering), and approaches using reinforcement learning. Recommendations need to be improved through "**exploitation and exploration**." "**Exploitation**" refers to recommending the product deemed optimal based on the model learned so far, while "**exploration**" refers to recommending a different product instead of the one considered optimal by the model, which is an approach based on reinforcement learning. These are high-level methods, so they will not be covered in this lecture, but if you're interested, please refer to the following references.

References:

- [Python Machine Learning] Python de hajimeru kyoushi nashi gakushuu (in Japanese)
- [Application of Reinforcement Learning to Recommender Systems] Suishin shisutemu e no kyouchiku gakushuu no tekiyou (in Japanese) (WSDM 2019)
- <https://research.google/pubs/pub47647/>

This concludes the basic explanation of the different types of recommendation methods. Each recommendation approach has its own merits and demerits. For example, collaborative filtering has the advantage of being highly effective and widely used by many companies. However, its drawbacks include the inability to recommend new products (since no one has bought them, there is no data) and the low recommendation accuracy for new customers (because they have not made any purchases and thus no data exists).

To offer better products and services to customers, research and development on hybrid approaches, deep learning, and reinforcement learning methods are actively being conducted.

Depending on the business situation, business model, marketing strategy, and data availability, it's essential to choose the right method. I recommend continuing your learning using the above references. When you encounter practical situations, apply the appropriate method in the right context.

✓ 11-2.2.5 Recommendation Evaluation Methods

As the final part of this section, I will briefly discuss recommendation evaluation methods. After building a recommendation model, it is necessary to integrate the recommendation logic into the actual service system (or provide recommendations directly) and verify whether it was effective.

For example, if you create a campaign webpage for a product or service, you need to collect data on whether the ad was displayed to users, whether users clicked on it, whether they signed up for a free trial, and whether they eventually made a purchase. (On sites like Amazon, data on products displayed to users, products clicked on, products purchased, and products reviewed should be stored.)

However, there is not just one metric to track when evaluating recommendations. For instance, the metric might be the website click-through rate, the percentage of shares on social media, or the actual conversion rate. The objectives vary. Another example is when recommending either product A or B. If the purchase prediction probability for A is 90% and B is 85%, it may be decided that recommending A is better. However, if B has a higher profit margin than A, it may actually be better to recommend B. In other words, recommendations often require multi-objective optimization.

As mentioned, the approach to recommendation evaluation depends on the purpose and logic of the recommendation. A commonly used method is **A/B testing**. This A/B

test is a verification approach for recommendations, where the better option is adopted. In a broader sense, it is one method of testing measures. In the next section, we will introduce methods for verifying measures, including A/B testing. We will also learn about more general approaches for evaluating the effectiveness of measures, not limited to web strategies.

✓ 11-2.3 Evaluation of Measure Effectiveness

Keywords: Causal Inference and Analysis, EBPM, Test Group and Control Group, RCT, A/B Test, Wilcoxon Rank-Sum Test, Selection Bias, Covariates, Intervention Variables, Regression Analysis

✓ 11-2.3.1 Correct Approach to Measure Effectiveness Evaluation

In this section, we will introduce several approaches to evaluating the effectiveness of measures. Measures can be related to policies, education, and various other areas. Recently, the keyword "**EBPM: Evidence-Based Policy Making**" has gained attention, which is also a data-driven decision-making approach. This section will primarily focus on the evaluation of the effectiveness of marketing measures. To maximize business objectives (such as profit) in line with a company's marketing strategy, it is important to quickly decide on business actions (variables) for that purpose, choose effective measures, and stop ineffective ones. Reducing wasted measures and increasing the success rate is essential, and this requires having robust evaluation methods for measures.

In week 11-1, we learned how to identify customers and segment them into valuable groups, emphasizing the importance of implementing measures for each customer segment. In the previous section, recommendations were introduced as one such measure. The next question that arises is, "Did this measure actually have an effect, or not?" How can we evaluate this? As mentioned in week 11-1, the key to analysis is "Method comparisons." However, the important aspect lies in the comparison targets. There are various ways to make comparisons, such as comparing sales before and after implementing a measure, or comparing the sales of customers who received the measure with those who did not. While these two approaches are simple and commonly used, they can actually lead to a misunderstanding of the measure's effectiveness.

✓ <Practice Question 11-2-2>

1. Why is it problematic to simply comparing sales before and after the implementation of a measure?
2. Why is it problematic to simply comparing sales between customers who received the measure and those who did?
3. How can an accurate comparison be made?

(Take some time to think about it yourself first, there will be an explanation provided below.)

First, let's consider the issue with the first approach of comparing sales before and after the measure. Various factors can come into play before and after the measure, making it difficult to determine if the measure itself had an effect. To illustrate with an extreme example, let's say a campaign was launched to sell masks on an e-commerce site between October and December 2019. If you compare the sales before and after the campaign, you might assume that sales dramatically increased after the campaign. However, the question is whether this can truly be attributed to the campaign's effectiveness. As you might expect, besides the campaign, other factors such as the seasonal increase in mask sales during winter and, furthermore, the outbreak of the coronavirus around January and February 2020, which also caused a surge in mask sales, should be considered. This example is extreme, so it's easy to recognize that the comparison is flawed, but it illustrates how other factors can make it difficult to accurately assess the effect of the measure itself.

Next, let's consider the issue with the second approach of comparing customers who received the measure with those who did not. In week 11-1, we performed decile analysis, so let's assume a measure where coupons are distributed to a specific segment. In this case, we are considering distributing coupons to Segment 10. Now, how should we verify the effectiveness of this coupon measure? Comparing the sales of Segment 10 before and after the measure would have the same issue as the previous example, but comparing them to other segments (1-9) also presents problems. This is because the people in Segment 10 already have a higher purchasing intent, so it is not an "apple to apple" comparison with the other segments. People in Segment 10 are already inclined to make purchases even without the coupon, so comparing them with segments that are less likely to purchase is not a valid comparison. In other words, it is not that the measure was effective; rather, there were other factors at play. This is essentially the same issue as the one mentioned earlier.

So, how should we make the comparison? There are several approaches, but the first step is to determine the ideal comparison group in order to make an **"apple to apple" comparison**. One approach is to divide the group into a test group (also called the treatment group) and a control group (also called the comparison group), targeting segments with the same purchasing power, which was briefly mentioned in week 11-1. This is called an **RCT (Randomized Controlled Trial)**, a method of randomized comparative testing. Specifically, continuing from the previous example, suppose there are 100,000 people in Segment 10, and we randomly select 90,000 of them to distribute the coupons, while doing nothing for the remaining 10,000. Since they are all from Segment 10, both the 90,000 who receive the coupon and the 10,000 who do not should have similar purchasing tendencies. This approach makes the comparison "apple to apple," allowing for a more accurate comparison.

The concept of RCT can initially be understood by imagining populations and samples, which are concepts you would learn in statistics. However, for those hearing about it for the first time, the explanation above might be a bit hard to visualize, so let me explain it using the following example of a coupon campaign.

Below is a time-series line graph showing the average sales per customer for both the test group and the control group, where customers with similar purchasing behavior at a supermarket or convenience store are divided into the test group and the control group. The test group is offered a coupon, while the control group receives no treatment. Before the campaign, both groups show similar sales trends. However, during the campaign, the average sales for the test group rise above those of the control group, and once the campaign ends, the sales for both groups fall and return to a similar trend.

```
test_gp =[21,15,10,20,21,15,30,40,41,42,20,41,26,10,15,20,18,17,21,22]
control_gp =[21,15,10,20,21,15,30,30,10,15,5,30,20,10,15,20,18,17,21,22]

plt.figure(figsize=(20,7))
plt.plot(test_gp,label='test group')
plt.plot(control_gp, linewidth=6,linestyle="--",label='control group')
plt.title('Test Group VS Control Group Avg Reveue')
plt.legend(prop={"size": 20}, loc="best")

plt.vlines(6, ymin=-1, ymax=50, colors='red', linestyle='dashed', linewidth=2)
plt.vlines(13, ymin=-1, ymax=50, colors='red', linestyle='dashed', linewidth=2)
plt.annotate("Coupon Campaign Period", xy = (7, 1), size = 20, color = "red")
plt.annotate("Before Campaign", xy = (2, 1), size = 20, color = "red")
plt.annotate("After Campaign", xy = (16, 1), size = 20, color = "red")
plt.xlabel('time')
plt.ylabel('avg revenue')
```

From the above results, it is clear that the sales increased due to the campaign. By considering the increase in sales and the actual cost of the campaign, it is also possible to calculate the **ROI (return on investment)** of the campaign. Specifically, the difference in sales between the test group and the control group is 100 yen (as the sales increment occurred only during the campaign period in the settings above).

```
sum(np.array(test_gp) - np.array(control_gp))
```

If we calculate simply, the effect of the campaign is an increment of 100 yen per person. If the cost per person is 80 yen, we can estimate the ROI as $ROI = 100 / 80 = 125\%$. (However, since the RCT has been strictly implemented here, we are able to make this simple calculation.)

Furthermore, if the campaign is conducted in a limited area, it is possible to estimate the overall effect. In practice, retailers and consumer goods manufacturers often conduct tests in specific regions to assess the effect and then decide whether to roll it out nationwide. (Another approach is to divide by store units, with test and control stores.)

On the other hand, in reality, there are cases where implementing an RCT is difficult. I understand these situations well, having seen many such cases. There may be costs associated with excluding controls, insufficient sample sizes, or ethical concerns about discriminating against customers. In marketing campaigns I've been involved with, there have been cases where no control was applied, and in practice, it can be quite challenging.

Even in cases where an RCT cannot be done, while the rigor may be lacking, there are several alternative approaches. In this course, we will introduce a few of these approaches. Additionally, the following reference discusses in detail the various approaches that can be taken in different situations, which will be helpful. A systematic and organized table of these approaches will be introduced toward the end of this section.

Reference:

- "Finding Causal Relationships for Policy Evaluation" Table A4 in A.3 Experiment and Quasi-experiment Methods

In the example above, ROI was calculated simply; however, calculating this metric without conducting an RCT is often not rigorous. The books below recommend tracking indicators such as "internal rate of return (the return on investment for implementing

campaigns or initiatives)" or "payback period (the time required to earn cumulative profits equal to the cumulative expenditures made)." While these are financial metrics and outside the scope of this course, those interested may refer to the following books for more information.

(From a data strategy perspective, the results of such initiatives are often summarized using PowerPoint or Excel. While these tools are not inherently bad, it is common to encounter inconsistencies in metrics or situations where manual effort is required later for integration. By establishing a system to store the results of initiatives as data, you can turn these insights into a strength for your company. Consider implementing infrastructure to accumulate these results for future use.)

Reference:

- 15 Key Metrics Everyone in Marketing Should Know: Data-Driven Marketing*

✓ Side Note 1: Reversal of Statistical Relationships Between the Whole and Parts

When causal estimation or analysis of initiatives is not conducted rigorously, there is a risk of making incorrect judgments. A common example illustrating this is **Simpson's Paradox**, which demonstrates that the effect observed in the overall data can differ from the effect observed in subsets of the data. Let's consider a concrete example.

Imagine the following table showing purchase rates. It represents the results of a campaign targeting a specific product, categorized by gender, showing whether individuals made a purchase. When looking at the purchase rates for males and females separately, the rates are higher for the group where the initiative was implemented. For males, the purchase rate is 95% in the initiative group compared to 90% in the non-initiative group. For females, the rates are 81% and 71%, respectively.

However, when looking at the overall purchase rates, the total for the initiative group is 84%, while for the non-initiative group, it is 85%. This result shows that the purchase rate is unexpectedly higher for those not exposed to the initiative. Why does this happen?

	Initiative Implemented			Initiative Not Implemented		
Gender	Purchased	Not Purchased	Purchase Rate	Purchased	Not Purchased	Purchase Rate
Male	100	5	0.952	300	30	0.900
Female	300	70	0.810	100	40	0.710
Total	400	75	0.842	400	70	0.850

When examining the subsets by gender, this initiative seems effective. However, when looking at the overall data, it appears ineffective. This is why it's called a paradox. Addressing this issue requires not only classical statistical approaches but also considering the "context behind the data" and the "causal factors that produced the results." As mentioned in the introductory section of this course, there are things that cannot be understood by looking at data alone, which relates to this discussion.

For example, suppose the product is a men's shampoo typically sold in pharmacies. While some women might purchase it, the product is primarily targeted at men, so women's purchase rates are generally lower. When looking at the gender ratio in the initiative group, women represent a higher proportion (105 men and 370 women). This means that, overall, the purchase rate for the initiative group decreases because the higher proportion of women, who are less likely to purchase, skews the overall rate lower than the randomly selected individuals in the non-initiative group.

This suggests that, in addition to the relationship "initiative → purchase," another factor, "gender → purchase," is at play. In such cases, it is essential to use data separated by gender.

The above example is based on the following book. For those interested in learning more, please refer to it:

Reference:

- Introduction to Statistical Causal Inference

✓ 11-2.3.2 Causal Inference with RCT

Now that we've covered the conceptual discussion, let's explore actual RCT data using Python. Below is data from an email marketing initiative conducted by an e-commerce site.

Target Data: Kevin Hillstrom MineThatData E-Mail Analytics Data Mining Challenge (2008)

http://www.minethatdata.com/Kevin_Hillstrom_MineThatData_E-MailAnalytics_DataMiningChallenge_2008.03.20.csv

First, let's load this data.

```
email_analytics_df = pd.read_csv('http://www.minethatdata.com/Kevin_Hillstrom_M
```

```
email_analytics_df.head()
```

Here is the translation for the data columns from left to right:

- recency: Months since the most recent purchase
- history_segment: Purchase segment from the previous year
- history: Purchase amount from the previous year
- mens: Flag indicating whether there was a purchase history for men's products last year
- womens: Flag indicating whether there was a purchase history for women's products last year
- zip_code: Area (based on the zip code), such as urban or suburban areas
- newbie: Flag indicating whether the user became a new customer within the last 12 months
- channel: Purchase channel
- segment: Type of email sent (male, female, or none)
- visit: Flag indicating whether the user visited the website within two weeks after receiving the email
- conversion: Flag indicating whether the user made a purchase within two weeks after receiving the email
- spend: Amount spent during the purchase

By checking the data below, we can see that there are 64,000 records.

```
email_analytics_df.info()
```

Now, to make it easier to understand whether the email campaign was implemented or not, we will process the data. Since the column related to the email campaign is 'segment,' we will aggregate by segment, which results in the following.

```
email_analytics_df.segment.value_counts()
```

We will add a column with a flag of 1 for 'Womens E-Mail' and 'Mens E-Mail,' and a flag of 0 for 'No E-Mail'.

```
email_analytics_df['flg'] = email_analytics_df["segment"].map(lambda x: 0 if x != 'Womens E-Mail' and x != 'Mens E-Mail' else 1)  
email_analytics_df.flg.value_counts()
```

At this point, we will calculate whether there is a difference in conversion rates and purchase amounts between the group that received the email (flg=1) and the group that did not receive the email (flg=0).

```
email_analytics_df.groupby(["flg"])[["conversion", "spend"]].mean()
```

From the above results, the conversion rate for the group that received the email is about 1%, while the group that did not receive the email has a conversion rate of about 0.5%, showing a difference of nearly 2 times. As for the purchase amount, the group that received the email has a value of 1.24, while the group that did not receive the email has a value of 0.65, again showing a difference of nearly 2 times. The difference in effect can be calculated as $1.24 - 0.65 = 0.59$, indicating that there is an effect of 0.59.

Now, let's test whether this difference is statistically significant. We will use the **Wilcoxon rank-sum test**, under the following assumptions. Since some statistical estimation terms (such as null hypothesis, alternative hypothesis, p-value, etc.) are used here, those who are not familiar with them should refer to materials from week 4.

Here is the translation for the supplementary note:

Supplement: Assumptions of the Wilcoxon Rank-Sum Test

- The means of the two groups are independent (there is no correspondence between the data).
- The data cannot be assumed to follow a normal distribution (non-parametric).
- The test is suitable for ordinal scale data.
- It is used to test the difference in central tendencies between two independent groups.
- The sample sizes of each group do not need to be equal.

```
from scipy import stats
s, pvalue = stats.mannwhitneyu(email_analytics_df[email_analytics_df["flg"]==1]
                               , email_analytics_df[email_analytics_df["flg"]==0]["spend"]
                               ,alternative='two-sided')
```

With a significance level of 0.05, based on the following test results, the null hypothesis ('there is no difference between these two groups') is rejected, and it is concluded that there is a difference between the two groups. Therefore, we can conclude that this campaign was effective.

```
pvalue < 0.05
```

<Practice Question 11-2-3>

Let's examine whether there is a difference between the two groups, Mens E-Mail and No E-Mail, using the above data. Next, check whether there is a difference in past purchasing power between the two comparison groups.

✓ Answer

The following is implemented in the same way as before, after applying the filter.

```
from scipy import stats
s,pvalue = stats.mannwhitneyu(email_analytics_df[email_analytics_df["segment"]=
                                , email_analytics_df[email_analytics_df["segment"]=="Mens E-Mai
                                ,alternative='two-sided'])
pvalue < 0.05
```

The same result was obtained as above.

Now, since this data assumes that an RCT has been conducted, robust methods have been used for campaign design and verification. However, if an RCT has not been conducted, is the way we are testing correct? As mentioned at the beginning, there is a possibility that emails were sent to people with higher purchasing power, which is called '**selection bias**.' In that case, since the target group originally had higher purchasing power, purchases may have been high regardless of whether the campaign was implemented.

Therefore, here we will examine the past purchase amounts for both the group that received the email and the group that did not receive the email.

```
email_analytics_df.groupby(["flg"])["history"].mean()
```

Since an RCT has been conducted, there likely isn't much difference in past purchase amounts, but let's test it just to be sure.

```
from scipy import stats
s, pvalue = stats.mannwhitneyu(email_analytics_df[email_analytics_df["flg"]==1]
                                , email_analytics_df[email_analytics_df["flg"]==0]["history"]
```

```
,alternative='two-sided')
pvalue < 0.05
```

From the results above, the null hypothesis is not rejected, and it can be concluded that 'there is no difference in past purchases between these two groups.' Since an RCT has been properly conducted, there appears to be no selection bias.

Reference: The answer explanation is above, but since this data can be aggregated by segment, let's explore the data a bit for better understanding. For example, for each past segment, we can calculate how many people received which type of email, as well as their past purchase amounts, conversion numbers, conversion rates, total purchase amounts, and average purchase amounts. (If you don't understand the implementation below, please review pandas.)

Looking at the aggregation results below, we can see that there is a difference between the group that received the email and the group that did not, for each past segment (7 segments based on purchase amounts).

```
aggregations = {"history":["mean"]
                 ,"conversion":["count","sum","mean"]
                 ,"spend":["sum","mean"]}
email_analytics_df.groupby(["history_segment", "segment"])[["history", "conversion", "spend"]].agg(aggregations)
```

Side Note 2: A/B Testing

This RCT is an approach commonly known as **A/B testing**. A/B testing has become standard in the internet industry (such as in online ads and design). Google, which provides search engine services, is said to conduct 1,000 A/B tests per day, and Microsoft reportedly performs 200 tests. For those interested in A/B testing, the following website and the recently released book (as of March 2021, the Japanese translation of A/B Test Practical Guide by ASCII) are very helpful.

Reference URL: <https://exp-platform.com/2019webconfabtutorial/>

Reference Book:

- A/B Test Practical Guide (ASCII)

In addition, outside of the internet industry, apparel company Workman, which was introduced in week 11-1, is also conducting large-scale A/B testing, and this has contributed to their sales growth.

Reference URL:

- Workman's Large-Scale Store Openings Behind A/B Testing: Data-Driven Management

Behind the growth of these companies is a culture of conducting numerous experiments by repeating A/B tests and quickly executing the PDCA cycle. It clearly shows that it is important for organizations to approach this as a company-wide initiative.

✓ 11-2.3.3 Causal Inference through Regression Analysis

Next, we will consider what to do when RCT is not feasible. Here, we introduce causal inference using regression analysis. (You can review about regression analysis and multiple regression analysis using materials from week 4 and 5, respectively.)

As discussed above, we talked about selection bias, and now we will attempt to remove it using regression analysis.

Regression analysis (multiple regression analysis) is a model that predicts the dependent variable (Y) using two or more independent variables (X). In the case of this email marketing, the dependent variable is the purchase amount (spend), and the independent variables are the presence of email delivery (flg) and past purchase amount (history). Here, the selection bias is represented by the "past purchase amount (history)," which is also called a covariate. The presence of email delivery is an **intervention variable**.

Purchase amount (Y: outcome) $\Leftarrow$ Email delivery (X: **intervention variable**), Past purchase amount (Z: covariate)

In this case, we want to determine whether the purchase amount (Y) is influenced by the intervention variable (email delivery), and the motivation is to reduce the effect of past purchase amount (the covariate or selection bias) as much as possible.

Following the analysis design above, we will perform multiple regression analysis. Here, we will use statsmodels instead of machine learning libraries like sklearn because statsmodels provides detailed output of the analysis results.

```
import statsmodels.api as sm

X = sm.add_constant(email_analytics_df[["history", "flg"]])
y = email_analytics_df["spend"]

model = sm.OLS(y, X)
result = model.fit()
```

Let's take a look at the results of the multiple regression model.

```
result.summary()
```

From the above results, the intercept is 0.3464, the coefficient for past purchase amount is 0.0013, and the coefficient for email send status is 0.5945. The t-tests for all of these have p-values less than 0.05, indicating that the coefficients are statistically significant.

Thus, we can conclude that the effect of sending the email increases sales by 0.59, which is consistent with the results from the previous RCT.

In this case, we performed multiple regression by adding one covariate in order to reduce the impact of selection bias, and by adding more covariates, we can bring the results even closer to the RCT results.

However, there is a problem: in some cases, this covariate cannot be obtained as data, making it impossible to fully eliminate selection bias. In such cases, there are several approaches (e.g., instrumental variable methods), which will be briefly introduced towards the end of this section. For those interested, further research on the topic in reference books is recommended.

Furthermore, when performing multiple regression analysis for effect verification, it is not about just adding any and all dependent variables and covariates. Instead, by using business context and domain knowledge to properly design the analysis, more robust and reliable verification can be achieved.

<Practice Question 11-2-4>

1. Perform multiple regression analysis on the two groups, Mens E-Mail and No E-Mail, as done above, and calculate the effect of the email.

✓ Answer

```
import statsmodels.api as sm

X = sm.add_constant(email_analytics_df[email_analytics_df["segment"]!="WoMens E
y = email_analytics_df[email_analytics_df["segment"]!="WoMens E-Mail"]["spend"]

model = sm.OLS(y, X)
result = model.fit()
result.summary()
```

You can see that the results are the same.

11-2.3.4 Summary of Policy Effect Verification Methods and Future Learning

So far, we have discussed the verification of policies, starting with simple before-and-after comparisons and their issues, the ideal method of RCT (Randomized Controlled Trials), and approaches using regression analysis when RCT is difficult to implement.

In addition to these, there are various other methods for policy verification. However, what might be concerning is the question: "Which approach should be chosen in the end?" The appropriate method can depend on the level of understanding of each approach, as well as the specific case at hand. While we haven't introduced every method, a systematic table for making such decisions is presented below.

	Method	
Strict Methods	1.RCT	Randomly assign subjects and
	2.Regression Discontinuity Design	Compare individuals around a
	3.Matching	Match similar individuals from
	4.Instrumental Variable Method	Use variables that influence p
	5.Difference-in-Differences / Fixed Effects Estimation	Exclude trend factors unrelate
	6.Synthetic Control Method	Combine data from non-subje
	7.Regression Analysis/ Simple Comparison	Use data after policy impleme
Simple but Less Strict Methods	8.Before-After Comparison	Compare results before and a

In the table above, explanations of each method along with their advantages and disadvantages are provided. The methods listed higher up are more rigorous, with RCT being the most robust approach. There are methods not covered in this lecture, so if you are considering applying them in practice, please refer to the following references and study them. Especially, I recommend reading at least the last chapter of Finding Causal Relationships for Policy Evaluation, even though some chapters include mathematical formulas.

References:

- Finding Causal Relationships for Policy Evaluation (Nihon Hyoronsha)
- Introduction to Statistical Causal Inference (Asakura Shoten)

There are also various approaches for policy evaluation, such as using structural equation modeling, Bayesian structural time series models, Bayesian networks for causal inference, and machine learning approaches like random forests. Additionally, causal exploration using deep learning GANs (SAM), reinforcement learning, and other advanced methods are also available. For those who might apply these in practice or are interested, they are introduced in the following references. I've also listed some libraries for causal inference. If you read these along with the books mentioned earlier, it should be sufficient for an advanced-level understanding.

References:

- Introduction to Effectiveness Evaluation (Gijutsu Hyoronsha)
- Causal Analysis with Python (Mynavi Publishing)
- Statistical Causal Inference (Asakura Shoten)
- Statistical Causal Exploration (Kodansha)

Reference Libraries: Python Libraries for Causal Inference

- DoWhy: A library that allows for the measurement of intervention effects based on observational data and causal knowledge between variables.
- EconML: A library specialized in measuring intervention effects in settings where variables are conditioned to influence a target variable.
- CausalML: A library focused on measuring intervention effects for specific attributes.
- CausalImpact: A library for time series data that predicts what would have happened if no intervention had been made, allowing for the measurement of the effect of an intervention when pre- and post-intervention data are available.

Before implementing any strategies, it is crucial to design the experimental plan in alignment with business and marketing strategies, execute the interventions, and establish a system within the organization to quickly iterate through the PDCA (Plan-Do-Check-Act) cycle. This approach is critical for ensuring continuous improvement. The following books, which focus on A/B testing in online environments, highlight the importance of rapid experimentation and continuous, incremental improvements. Even large companies like Google and Microsoft emphasize the importance of sustained efforts over time. For those interested, these references provide valuable insights.

Reference:

- "Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing" A/B Test Jissen Guide (in Japanese) (ASCII)

11-2.4 Creating BI Tools and Reporting with Python, and Automating the Process

Keywords: Automation of analytical reporting, automation process, integration with various tools, Plotly, Dash, email sending, Slack integration, PDF file creation

11-2.4.1 Automating Analysis Results and Report Creation (Creating Reporting and BI Tools)

In this section, we will introduce methods for automating reporting and creating BI tools for analysis results. After conducting an analysis and implementing strategies or service deployments, the next step is measuring their effectiveness. We will cover how to report these results, how to create BI tools for regular KPI checks, and how to integrate with other tools for notifications.

When performing tasks such as aggregating and visualizing KPIs set in a marketing strategy or checking analysis results, doing them manually every time or coding repeatedly or creating reports on the fly can consume a lot of time. Once the numbers and specifications to track as KPIs are decided, we will monitor their progress over several months to half a year, and the aggregation and reporting will become routine tasks. Therefore, tasks such as data manipulation, reporting, visualization, and notifications should be automated as much as possible. This way, we only need to do a final check on the numbers and graphs, and we can spend time analyzing areas for improvement. Let Python handle the parts that can be automated.

In this section, we will introduce the creation of simple BI tools and report generation (including PDFs), processing in Excel and Google Sheets, as well as email sending and integration with Slack.

11-2.4.2 Before Creating Automated Reports, BI Tools, and Dashboards

Before automating reporting or creating BI tools, there is an important point to consider. This is similar to what was discussed in week 11-1: clarifying the purpose of the dashboard or report and identifying what you want to learn from it.

With the rise of BI tools, many companies are adopting them, but there are cases where a BI tool is built, only to end up with many graphs that don't actually help clarify what needs to be done. It's essential to understand what you want to check using these graphs, what action you want to take based on the information, and how it fits into the business decision-making process or operational flow. Without this, the BI tool may eventually become underutilized. While practicing for implementation, like in this lecture, is fine, in real-world applications, we should aim to create reports and BI tools that lead to business actions.

✓ 11-2.4.3 BI Tools and Dashboards Created with Python

First, let's introduce the libraries Plotly and Dash, which allow interactive data visualization. Libraries like Matplotlib work well for basic data visualization. However, if you want to explore specific subsets of data, such as switching graphs by year or category, you would need to rewrite the code each time. Of course, with the right design, it is possible to implement it that way. However, if you want to quickly change the data and update the graphs accordingly, it takes more time.

By using Plotly and Dash, you can easily create BI tools where data changes by year or category, and the graphs adjust automatically. Additionally, you can quickly explore the data, changing the graphs and visualizing the results. As we have discussed, it is important to look at data carefully, but visual representations like graphs can often help with quicker understanding. In this section, we will explain Plotly, which allows for interactive graph manipulation, and Dash, which can be used to build web applications. Once you get familiar with these tools, publishing your data visualization results on the web becomes easy, and the implementation is simpler compared to other programming languages. Now, let's look at the implementation.

To update Plotly in your environment, you can use the following command in your terminal or command prompt:

```
!pip install --upgrade plotly
```

Import the necessary libraries below.

```
import plotly
import plotly.graph_objects as go
import pandas as pd
```

```
from datetime import datetime
import datetime
```

The following imports are for preparing the data used for data visualization. It will be stock price data.

```
import pandas_datareader.data as web
```

We will retrieve Google's stock price data from January 1, 2010, to December 31, 2020, using the Pandas `DataReader`. The start and end dates, along with the stock data for the target company, are specified to fetch approximately 11 years of stock data.

```
st_dt = datetime.datetime(2010, 1, 1)
end_dt = datetime.datetime(2020, 12, 31)
google_df = web.DataReader('GOOG', 'stoq', st_dt, end_dt)
```

```
google_df.info()
```

```
google_df.head()
```

To visualize the stock price data, we can use matplotlib to create time series charts that show trends over time. In this case, we will use Plotly to display the candlestick chart, which allows us to simultaneously view the opening, closing, high, and low prices for each day. The chart is commonly known as a candlestick chart. The high and low values vary from day to day, and on days with significant price movements, the high and low values will be more spread apart. By using the candlestick chart, we can better understand these fluctuations.

The following implementation demonstrates how to create the candlestick chart.

```
fig = go.Figure(data=[go.Candlestick(x=google_df.index,
                                     open=google_df['Open'],
                                     high=google_df['High'],
                                     low=google_df['Low'],
                                     close=google_df['Close'])])

fig.update_layout(
    title_text="Candle Google Stock"
)
```

By configuring the `rangeselector` as shown below, you can change the graph to display data from 1 month ago, 6 months ago, or 1 year ago. You can change the view by pressing the buttons located at the top-left of the graph.

```
fig.update_layout(
    xaxis=dict(
        rangeselector=dict(
            buttons=list([
                dict(count=1,
                    label="1m",
                    step="month",
                    stepmode="backward"),
                dict(count=6,
                    label="6m",
                    step="month",
                    stepmode="backward"),
                dict(count=1,
                    label="1y",
                    step="year",
                    stepmode="backward"),
                dict(step="all")
            ])
        ),
        rangeslider=dict(
            visible=True
        ),
        type="date"
    )
)
```

When you hover over the graph above, you can modify the range of the graph. Additionally, you can change the size or save it as an image. Please try experimenting with these features.

It is also possible to save the graph above as an HTML file.

```
# html file
plotly.offline.plot(fig, filename='output.html')
```

You can download the file from the left folder, or use the following code to specify the file name and execute it to download the file from Google Colaboratory to your local machine.

```
from google.colab import files
files.download('output.html')
```

As you saw earlier, with Plotly, you can interactively manipulate graphs. Next, we'll use Dash, which is also useful for web application development. We'll install the necessary packages. To use Dash in this environment, we'll install `jupyter_dash`.

```
!pip install dash
```

```
!pip install jupyter_dash
```

Next, we will process the data to be visualized. This is the same corporate POS data from the e-commerce site covered in week 11-1. Since the preprocessing steps are the same, I will skip the explanation.

```
# Read Excel data from the UCI website
# Download the data from the UCI website
!wget "http://archive.ics.uci.edu/static/public/352/online+retail.zip"
# Extract the download zip file
!unzip "./online+retail.zip"
# read the extracted Excel data
file_url = './Online Retail.xlsx'
all_online_retail_data = pd.ExcelFile(file_url)
online_retail_data_table = all_online_retail_data.parse('Online Retail')
online_retail_data_table = online_retail_data_table[(online_retail_data_table.CustomerID > 0)]
online_retail_data_table.CustomerID = online_retail_data_table.CustomerID.astype(int)
online_retail_data_table['TotalPrice'] = online_retail_data_table.Quantity * online_retail_data_table.UnitPrice
online_retail_data_table.set_index(['InvoiceDate'], inplace=True)
```

Next, we will define the items we want to check. In this case, we will look at monthly sales by country. Below, we use pandas' resample function to calculate sales by country and by month.

```
country_totalp_m_df = online_retail_data_table.groupby(["Country"])[['TotalPrice']]
country_totalp_m_df.head()
```

Of course you can graph all of these at once using Matplotlib, but it would become cluttered and hard to read. Therefore, let's use Plotly and Dash, as mentioned earlier, to allow the user to select a country from a dropdown. We will import the necessary libraries.

```
import dash
from jupyter_dash import JupyterDash
import dash_core_components as dcc
from dash import html
import plotly.express as px
from dash.dependencies import Input, Output
```

By running the following, a monthly sales trend will be displayed, and by changing the country, the time-series sales graph will be updated. Australia is set as the default. You can create interactive graphs that run in a web browser without knowing web languages such as HTML or CSS.

```
app = JupyterDash(__name__)
app.layout = html.Div([
    dcc.Dropdown(id="my_dropdown",
                 options=[{"value": cnt, "label": cnt} for cnt in countries],
                 value="Australia"),
    dcc.Graph(id="my_graph")
])

@app.callback(Output("my_graph", "figure"), Input("my_dropdown", "value"))
def update_graph(selected_country):
    selected_country_totalp_m_df = country_totalp_m_df[country_totalp_m_df["Country"] == selected_country]
    return px.line(selected_country_totalp_m_df, x="InvoiceDate", y="TotalPrice")

app.run_server(mode="inline")
```

A brief explanation of the `@callback` for those who may not be familiar: A callback is a process used to call another function when a specific event occurs. For example, when an element is clicked, the graph may be updated. In the example above, the function defined as `update_graph` has `selected_country` as an argument. If you look at the contents of the function, you will see that it filters based on a specific country.

Next, let's visualize the monthly sales data using a world map, as each country has its own data. Although it's not strictly necessary to create a world map for this case since the regions are limited, viewing the world map allows us to understand regional imbalances and sales trends. Let's approach this as a practice exercise.

Below is the data included in Plotly's express module:

It contains the country name, continent name, year of data collection, life expectancy, population, GDP per capita, iso_alpha (country abbreviation), and iso_num (country

code number). Since we want to recognize countries by their abbreviations, we will combine this data with the monthly sales data from earlier.

```
gapminder = px.data.gapminder()  
gapminder.head()
```

To combine this data with the previous sales data, we will rename the data columns.

```
country_totalp_m_df.columns = ["country","invicedate","totalprice"]
```

The following processes filter the data to records for each specific country.

```
new_information_tb = gapminder.groupby(["country","continent","iso_num","iso_al  
new_information_tb
```

The data is merged below. Additionally, there are some NA records, which will be temporarily excluded and deleted.

```
merge_df = country_totalp_m_df.merge(new_information_tb, how='left',on="country
```

```
merge_df.dropna(inplace=True)  
merge_df
```

Next, as we saw earlier, the sales for the UK are too large, so we will exclude the UK for now. Additionally, some sales are negative, so those will also be excluded.

```
new_merge_df = merge_df[(merge_df.totalprice>0) &(merge_df.country!="United Kir
```

Next, we will change the format of the year and month display by modifying the data type.

```
new_merge_df["YYYYMM"] = pd.to_datetime(new_merge_df["invicedate"]).dt.strftime
```

This concludes the data preprocessing. To create the world map, we will use `scatter_geo`, setting the relevant data, columns to categorize, and the size of the bubbles to represent sales.

The following is a world map where the size of each bubble represents the sales of each country, and the regions are color-coded.

```
fig = px.scatter_geo(new_merge_df, locations="iso_alpha", color="continent",
                    hover_name="country", size="totalprice",
                    animation_frame="YYYYMM",
                    projection="natural earth")

fig.show()
```

By pressing the play button on the graph above, you can view the monthly results as an animation. Just like when using Plotly, you can also zoom in and out, save the image, and more.

This concludes the brief introduction to creating a BI tool-like dashboard. While commercial software and cloud services make data visualization easier, implementing it with Python is not that difficult. You don't need to buy expensive software, and Python allows for detailed data processing and visualization that may not be possible with existing BI tools. By developing such visualization tools in-house, modifications can be made easily, enabling faster execution of the PDCA cycle.

<Practice Question 11-2-5>

1. Using the gapminder data loaded above, create a world map like the one shown above (with bubble sizes based on life expectancy), and make it interactive. Additionally, create a graph where the bubble sizes represent GDP per capita or population instead.

✓ Answer

```
gapminder.head()
```

Average life expectancy

```
fig2 = px.scatter_geo(gapminder, locations="iso_alpha", color="continent",
                    hover_name="country", size="lifeExp",
                    animation_frame="year",
```

```
projection="natural earth")  
  
fig2.show()
```

GDP per capita

```
fig2 = px.scatter_geo(gapminder, locations="iso_alpha", color="continent",  
                      hover_name="country", size="gdpPercap",  
                      animation_frame="year",  
                      projection="natural earth")  
  
fig2.show()
```

Population

```
fig2 = px.scatter_geo(gapminder, locations="iso_alpha", color="continent",  
                      hover_name="country", size="pop",  
                      animation_frame="year",  
                      projection="natural earth")  
  
fig2.show()
```

✓ Side Note 3: Use a map

If you want to use a detailed map or need flexibility in zooming in and out, folium is a useful library to use.

Official Website: <https://python-visualization.github.io/folium/>

```
!pip install folium
```

We will display a map around Kuala Lumpur International Airport. The latitude and longitude of Kuala Lumpur International Airport are (2.74558, 101.70992), so we will set this as the parameter for the location. Also, to add a marker to the Airport, we will use the `Marker`.

```
import folium  
  
# Creating the map and adding a marker  
folium_map = folium.Map(location=[2.74558,101.70992], zoom_start=12)  
folium.Marker(location=[2.74558,101.70992]).add_to(folium_map)  
  
# Displaying the map  
folium_map
```

This detailed map has a very fine level of granularity, so it may not be used as often in situations where a rough visualization is required for analysis. However, as shown below, you can use `PolyLine` to mark routes, which might be useful for tasks such as calculating the shortest distribution route.

```
loc = [(2.74558,101.70992),  
       (3.1579,101.7117)]  
  
folium.PolyLine(loc,  
                color='red',  
                weight=12,  
                opacity=0.8).add_to(folium_map)  
  
folium_map
```

11-2.4.4 Manipulating Google Spreadsheet with Python (Security Warning)

Note: Please be aware that from this point on, if you do not have a Google account or if you are restricted due to company security policies, you may not be able to execute some parts of the process. Also, since you will be handling security information, make sure to follow your organization's security policies and handle password information with utmost care.

Next, let's introduce how to manipulate Google Spreadsheet using Python. As for Excel operations, you should be able to handle data processing relatively easily when combined with Pandas operations, as covered in week 11-1. In recent years, many companies have started using Google Spreadsheet in the cloud in addition to Excel. Here, we will introduce how to directly process Google Spreadsheet using Python. By learning this technique, you can automate tasks such as file operations (numeric processing in sheets) that were previously managed on Google's cloud.

First, let's install the necessary package for this purpose.

<https://qiita.com/safa/items/bfa52430f920ac562bec>

```
!pip install gspread
```

The following are the necessary libraries and setups.

Once you run everything, a window for authentication will appear.

```
from google.colab import auth
auth.authenticate_user()

import gspread
from google.auth import default
creds, _ = default()

gc = gspread.authorize(creds)
```

The following code displays a list of files in your Google Drive account. In the author's environment, the "pass" file containing password information and a sample data file named "sample_data_2021" are already prepared for use.

```
workbook_list = gc.openall()
for workbook in workbook_list:
    print(workbook.title) # Display the titles of the retrieved files
```

The following code specifies a Google Spreadsheet and processes it.

```
ss_name = "sample_datasheet"
workbook = gc.open(ss_name)
```

```
worksheet = workbook.get_worksheet(0)
```

```
print(worksheet.acell("C2").value)
```

You can also specify the spreadsheet ID or URL and process it, as well as write to or delete from sheets. If there are numbers managed in Google Spreadsheet that need to be aggregated or processed regularly, you can use Python and gspread, so consider using this approach.

✓ 11-2.4.5 Creating PDF Reports with Python

After organizing numerical data and creating graphs, the next step is to compile them into a report. In this case, we'll introduce PDF output using the reportlab package. Below is the official website for the package.

Official Website: <https://www.reportlab.com/>

Then, you can create a sample PDF report like the following.

https://www.reportlab.com/media/pix/RLIMG_04e546a82cc44dbc9ac6699aa7572865.PD

E

Install reportlab.

```
!pip install reportlab
```

Next, we will import the necessary libraries.

```
from reportlab.pdfgen import canvas
from reportlab.pdfbase import pdfmetrics
from reportlab.pdfbase.cidfonts import UnicodeCIDFont
from reportlab.lib.units import cm
from reportlab.lib.pagesizes import A4, portrait
from PIL import Image
from matplotlib.backends.backend_pdf import PdfPages
import datetime
```

The following is the implementation that retrieves the current date and time and outputs it as a final PDF report.

```
# Get the current date
d_today = datetime.date.today()

### Generate PDF file ###
file_name = 'Report'+str(d_today)+'.pdf'    # File name
pdf = canvas.Canvas(file_name, pagesize=portrait(A4))    # A4 size
pdf.saveState()    # Save the state

pdf.setAuthor('OFFICE54')
pdf.setTitle(str(d_today))
pdf.setSubject('TEST TEST TEST')

### Draw a line ###
pdf.setLineWidth(1)
pdf.line(10*cm, 25*cm, 10*cm, 1*cm)

### Set font and size ###
pdf.setFont('Helvetica', 12)

### Draw tesxt ###
pdf.drawString(1*cm, 26*cm, str(d_today))

pdf.setFont('Helvetica', 20)    # Change font size
width, height = A4    # A4 size
```

```
pdf.drawCentredString(width / 2, height - 2*cm, 'Today\'s KPI Status and Past 1  
pdf.save()
```

The following confirms that the PDF file created above exists.

```
ls
```

The file name will be "ReportYYYY-MM-DD.pdf," and it is also possible to download the created PDF file to your local machine.

```
from google.colab import files  
files.download(file_name) # Specify the file variable name
```

The creation of the PDF file has been completed.

✓ 11-2.4.6 Automating Notification Features in Python (Security Warning)

For report generation tasks, depending on the amount of data, preprocessing, and the volume of the report, the process may take some time. One way to automate these tasks is by using tools like the Windows Task Scheduler or Linux cron to create batch processes for daily or weekly executions. While you can estimate the processing time for report generation to some extent, it is cumbersome to manually check if the process has been completed. Therefore, here we introduce a way to automatically notify when the process has been completed. You can send notifications via email or services like Slack, Line, or Teams. In this case, we will demonstrate how to use email and Slack for notifications.

However, since security-related settings are required, only the methodology will be introduced here. If you need to implement this in a business setting, please be cautious with security measures and execute it accordingly.

✓ 11-2.4.6.1 Notification via Email and File Attachment

Here, I will introduce a method using Gmail. Please note that you need to turn on the option "Allow less secure apps to access your account" in your Google account settings. Since this poses a security risk, it is recommended to create a separate account for learning purposes.

To enable "Allow less secure apps to access your account," go to the Google account management page, scroll down to the "Less secure app access" section under security, and select "Turn on" to enable "Allow less secure apps: On."

We will import the necessary libraries for sending emails. MIME stands for "Multipurpose Internet Mail Extensions," which is a specification for handling email data over TCP/IP networks. It allows handling data beyond ASCII characters, such as images and attachments.

```
from email.mime.text import MIMEText
from email.mime.application import MIMEApplication
from email.mime.multipart import MIMEMultipart
import smtplib
from os.path import basename
```

Here, you will set up the email account information and specify the recipient's email address. Instead of directly writing the password, we will read the password from a file stored on Google Drive.

```
from google.colab import auth
auth.authenticate_user()

import gspread
from google.auth import default
creds, _ = default()

gc = gspread.authorize(creds)
```

```
# Get password
ss_name = "pass"
workbook = gc.open(ss_name)
worksheet = workbook.get_worksheet(0)
password = worksheet.acell("A1").value
```

```
# SMTP authentication information
account = "test.gci@gmail.com"
```

The sender and recipient are set below.

```
# Sender and recipient
to_email = "test.gci@gmail.com"
from_email = "test.gci@gmail.com"
```

Next, set the email title (subject), email body (message), and configure the recipient and sender.

```
# Creating MIME
import datetime

subject = "Test Email"
text = " Nice to meet you. I look forward to working with you."
message = "This is a test email:" + str(datetime.datetime.today()) + text
msg = MIMEText(message, "html")
msg["Subject"] = subject
msg["To"] = to_email
msg["From"] = from_email
```

The email will be sent with the following.

```
# Mail sending process
server = smtplib.SMTP("smtp.gmail.com", 587)
server.starttls()
server.login(account, password)
server.send_message(msg)
server.quit()
```

Note: If not enabled, "Blocked insecure apps" will appear.

This completes the email sending process.

Next, to attach a file to the email, implement it as follows. Let's attach the PDF file created earlier.

```
# MIME creation
subject = "Report Creation Email" + str(datetime.datetime.today())
message = "Here is today's report" + str(datetime.datetime.today())
msg = MIMEText(message)
msg["Subject"] = subject
msg["To"] = to_email
msg["From"] = from_email
msg.attach(MIMEText(message))

# Attach the PDF file (HTML files, etc., can also be attached)
path = "." + file_name
with open(path, "rb") as f:
    part = MIMEApplication(
        f.read(),
        Name=basename(path))
```

```

    )

    part['Content-Disposition'] = 'attachment; filename="%s"' % basename(path)
    msg.attach(part)

    # Sending the email
    server = smtplib.SMTP("smtp.gmail.com", 587)
    server.starttls()
    server.login(account, password)
    server.send_message(msg)
    server.quit()

```

When checking the email, you can see that the PDF file created earlier is attached and sent. Of course, the HTML file of the Google stock price time series trend created in section 5.3 can also be attached and sent via email.

✓ 11-2.4.6.2 Notification via Slack

Next, let's post to Slack using Python. The steps are as follows:

1. In Slack settings, choose the channel to post to under Incoming Webhook (Post to Channel) and click "Add Incoming Webhook integration."
2. A Webhook URL will be displayed, so copy it.
3. Click "Save Settings."

To post to Slack from Python, we need to install slackweb.

```
!pip install slackweb
```

The obtained URL will be saved in a file on Google Drive, and it will be read from that file.

```

import slackweb

# Obtain the URL
slack_url = worksheet.acell("A2").value

```

```

slack = slackweb.Slack(url="https://hooks.slack.com/services/"+slack_url)
slack.notify(text="Posting to Slack from Python. Today's date is "+str(datetime

```

The OK message is displayed above, and when checking Slack, it is confirmed that the message has been posted.

This concludes the explanation on posting to Slack using Python. You can also attach files to posts on Slack, and similarly, using different libraries, you can post to other platforms such as Line and Microsoft Teams. If you're interested, you can look into it further.

This concludes the explanation on creating BI tools, generating reports, and posting notifications via email or Slack after running an analysis. These individual techniques may not be very meaningful on their own, but as mentioned at the beginning, by combining these techniques to design and implement a marketing system, operational load can be reduced. The key is not just to create the numbers or reports themselves, but to look at the metrics that lead to improvements in management and planning, and how quickly you can move the PDCA cycle towards the next action. As demonstrated in this chapter's implementation, it is not particularly difficult, so you can consider developing it in-house. Even if you outsource it, if you understand the necessary implementations to some extent, it will help you achieve better business results.

Reference:

- [Python Interactive Data Visualization Introduction] Python Interactive · Visualization Nyuumon (in Japanese)

<Practice Question 11-2-6>

1. Design and implement an automated process for generating analysis reports using the libraries mentioned above.

✓ Side Note 4: Periodic Execution using Python

As explained above, periodic execution can be achieved using cron on Linux, but it is also possible to execute it using Python's crontab library or the schedule library. Here, we will introduce schedule.

```
!pip install schedule
```

The following will display the current date and time as the job function to be processed.

```
import schedule
import datetime
```

```
import time

def job():
    print(datetime.datetime.now())
```

Next, we will schedule this to run every 10 seconds and set it up as shown below. The "seconds" after "every" represents seconds, but it can be changed to minutes, hours, etc.

```
schedule.every(10).seconds.do(job)

while True:
    schedule.run_pending()
    time.sleep(1)
```

The above will run in an infinite loop, so to stop the calculation, please press the stop button.

✓ 11-2.5 The Next Steps in Consumer Analysis

Keywords: Business Data Analysis Projects, Advanced Approaches in Marketing Analysis

✓ 11-2.5.1 Before Advancing a Data Analysis Project in the Field

This is the final section of the advanced consumer analysis. In this course, we have focused on Python implementations for consumer analysis, with the aim of allowing you to get hands-on experience. As mentioned in week 11-1, in actual business settings, it is essential to consider not just programming and technical aspects but also corporate strategies and various perspectives. There are many things to think about in a real project, such as the business background, the challenges at hand, the analysis goals, the value of building predictive models, the target data, and how to manage the operation. Simply being able to implement Python does not automatically increase the value of data analysis, nor does it necessarily improve customer service. It is crucial to design the analysis appropriately, implement it, and consider business impact estimation, execution, validation, and operations, and then work on improvements using a typical PDCA (Plan-Do-Check-Act) cycle. Moreover, it is essential to accumulate the new result data from CRM (Customer Relationship Management) as part of the process. Through this lecture, I hope you have gained a basic understanding of how to implement

various steps (from data loading and exploration to model building and validation, and automation) with Python, and how to build a comprehensive flow from start to finish.

The Best Way to Gain Data Analysis Skills

The best way to gain data analysis skills is by experiencing it in the field. However, if you jump straight into a real project using actual data without a clear objective, there's a high likelihood that the project will fail or become difficult to push forward. Therefore, as a first step, I recommend practicing by imagining a hypothetical project and using sample data like the one below to consider what kind of analysis can be done, and what kind of business impact it could have. Although it's hypothetical, there are many factors to think about. Analyzing data, implementing data analysis, building the necessary IT infrastructure for future operations, and envisioning business expansion are all excellent exercises as case studies. By considering not only Python implementation, data aggregation, and model building but also how to advance an actual data analysis project, you'll enhance your practical skills. It's important not just to solve the given problem, but also to develop the ability to ask your own questions.

Using the following sample data, think about how you could expand the business, what challenges you should address, and what proposals you could make to management and decision-makers. Please also refer to the framework below for structuring your report presentation.

Supplemental Note: As referenced in the following Google paper, even when building machine learning systems, model construction is just a small part of the process. At the business level, the scope of consideration expands further, so it is necessary for the company to approach data strategically, valuing it across the entire organization and incorporating it into the broader business strategy.

Reference Paper:

- Hidden Technical Debt in Machine Learning Systems
- <https://papers.nips.cc/paper/2015/file/86df7dcfd896fcdf2674f757a2463eba-Paper.pdf>

References:

- [Building Machine Learning Powered Applications: Going from Idea to Product]
- [Working with Machine Learning: Starting with Practical Applications, 2nd Edition]
Shigoto de hajimeru kikai gakushu (in Japanese)

✓ Sample Data for a Virtual Analysis Project

1. Data for Loan (Lending) Business

- https://www.openintro.org/data/csv/loans_full_schema.csv
- <https://www.kaggle.com/c/home-credit-default-risk>

2. Data for Bike Sharing Business

- <https://archive.ics.uci.edu/ml/datasets/Bike+Sharing+Dataset>
- <https://www.capitalbikeshare.com/system-data>

3. Public Data Available on UCI's Website

- <https://archive.ics.uci.edu/datasets>

✓ Framework for Presentation Report of a Virtual Analysis Project

1. Business Background and Analysis Purpose
2. Analysis Policy and Design
3. Description of the Target Data
4. Report on Basic Analysis and Exploratory Analysis of the Target Data
5. Purpose, Evaluation, and Report of Model Construction
6. Points of Improvement in the Model, and Innovations
7. Proposals for Measures and Business Actions, Expected Effects, and Estimations
8. Current Issues
9. Future Directions
10. Appendix (Supplementary Information, etc.)

✓ 11-2.5.2 More Advanced Analysis Approaches

Finally, we will discuss more advanced topics. There are various methods for consumer analysis, and while this course did not cover them all, here are some advanced techniques and motivations.

- Hierarchical Bayesian: Consumers are different from each other, and we want to model these differences by considering their heterogeneity.
- Bayesian Structural Time Series Model: While Randomized Controlled Trials (RCTs) are not available, there are time-series data with both intervention and non-intervention periods. We want to estimate causality using these data (Note: One Python package for this is CausalImpact, which was introduced earlier).

- Structural Equation Modeling: We want to handle latent variables that cannot be captured by regression analysis and other traditional methods.
- Factor Analysis: Used for analyzing brand evaluations or survey data.
- Price Setting (Dynamic Pricing): We want to understand how to set prices and determine the optimal price.
- Demand Forecasting: We aim to manage product production without holding excess inventory and avoiding stockouts, aligning with marketing strategies to ensure appropriate production management.

For those interested in these techniques, please refer to the following references or study them through related books or courses. In consumer analysis, there are situations where machine learning methods may not adapt well or are difficult to apply. In such cases, these methods can be useful, so consider them as an option. However, if there are other business problems to solve, their priority may decrease, and there may not be a need to focus too much on advanced approaches. In my experience with analysis-related projects, these techniques were used in less than 10-20% of cases, and more time was often spent addressing various challenges before considering advanced methods.

References:

- [Bayesian Statistical Modeling with Python: A Practical Guide to Data Analysis with PyMC] Python ni yoru beizu modeling: PyMC deno data bunseki guide (in Japanese) (Kyoritsu Shuppan)
- [Experiencing Bayesian Inference with Python: An Introduction to MCMC Using PyMC] Python de taiken suru beizu suiron: PyMC ni yoru MCMC nyumon (in Japanese)(Morikita Publishing)
- [Introduction to Statistical Modeling for Data Analysis: Generalized Linear Models, Hierarchical Bayesian Models, and MCMC (Probability and Information Science)] Data kaisekino tameno toukei modeling nyumon:Ippanka senkei model · kaisou beizu model · MCMC(kakuritsu to jouhouno kagaku)(in Japanese)(Iwanami Shoten)
- [Practiac Marketing Research and Analysis with R] R ni yoru jissenteki marketing research to bunseki (in Japanese) (Kyoritsu Shuppan)

- [AI Algorithms in Marketing: Machine Learning/Economic Models for Automation, Best Practices, and Architecture] AI algorithm marketing jidouka no tameno kikai gakushu/keizai model, best practice architecture(in Japanese) (Impress)
- [The Price Rule] Kakaku no okite (in Japanese) (Chuo Keizai Sha)
- [Fundamentals of Demand Forecasting:Revolutionizing SCM and Marketing] Juyou yosoku no kihon SCM to marketing wo gekiteki ni kaeru (in Japanese) (Nihon Jitsugyo Publishing)

Acknowledgment: For use of the following two datasets